Computer Network and Communication
CNS3200

Learning Outcome

- **EXPLAIN THE MECHANISM & SHOW THE CALCULATION OF ERROR CONTROL AND THE RELATED PROTOCOLS (NS)**

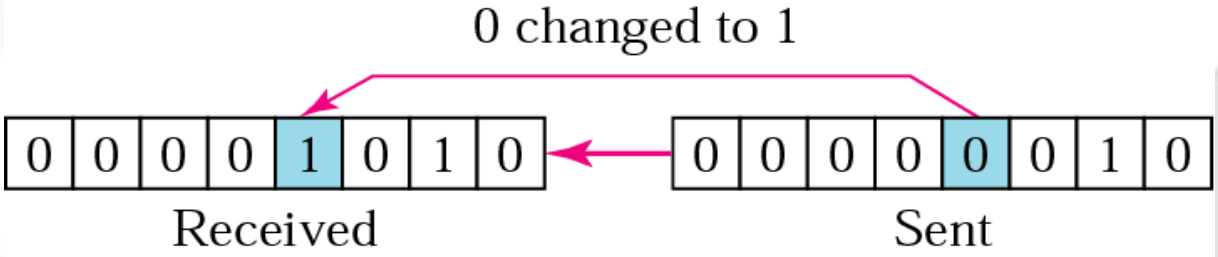
Contents

Data Link Layer

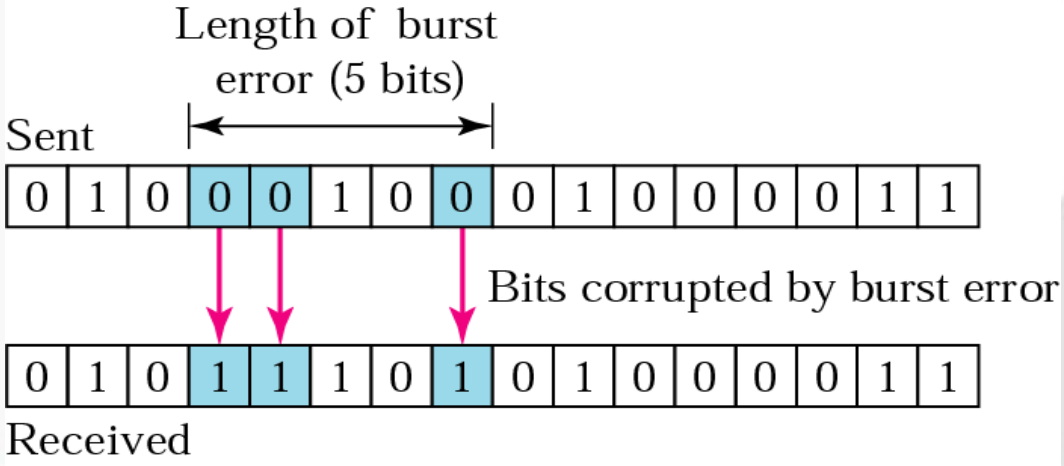
- Types of Error
  - Single bit error
  - Burst error
- Error Detection
  - Parity Check
  - CRC
  - Checksum
- Error Correction
  - Hamming Code

Types of Errors

Single bit error – only one bit is changed from 1 to 0 or from 0 to 1.

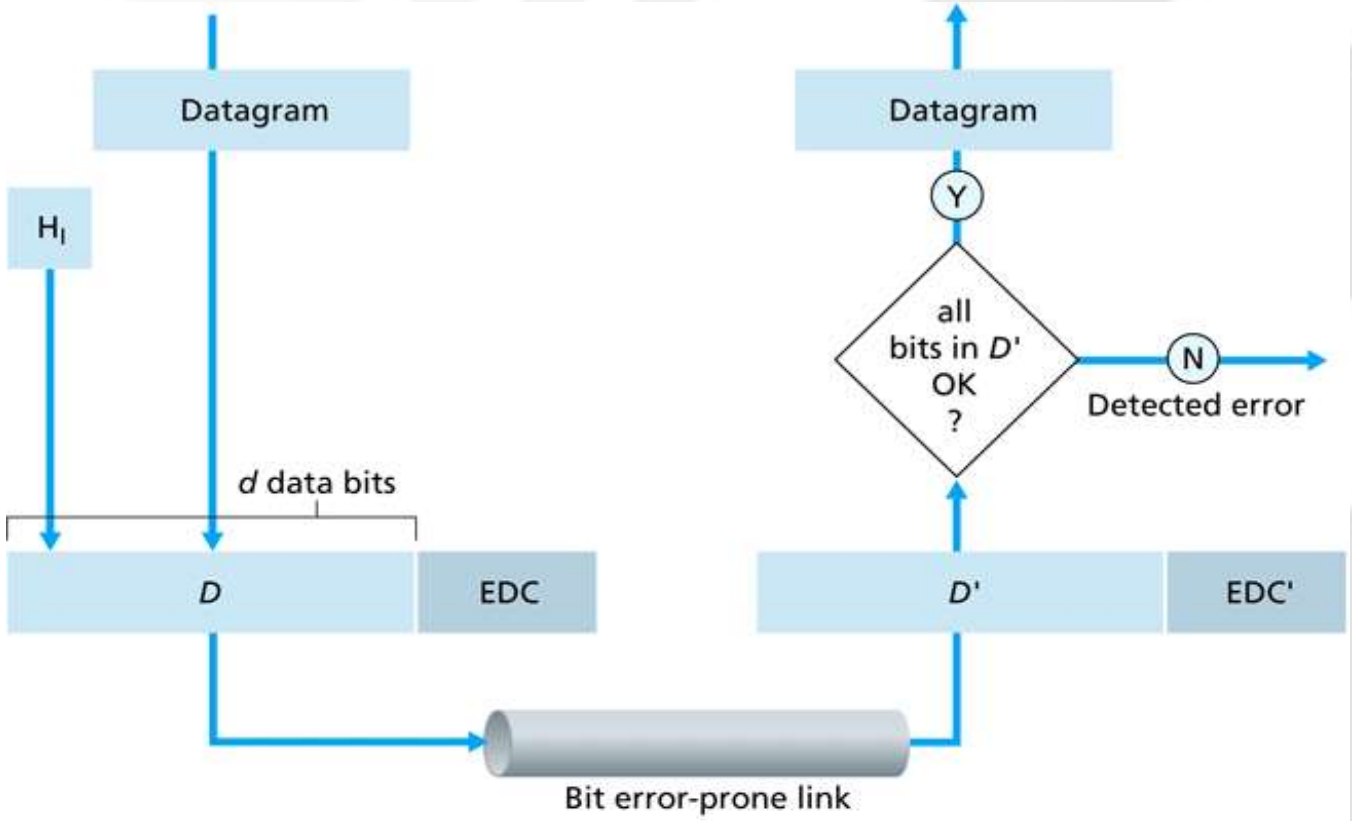


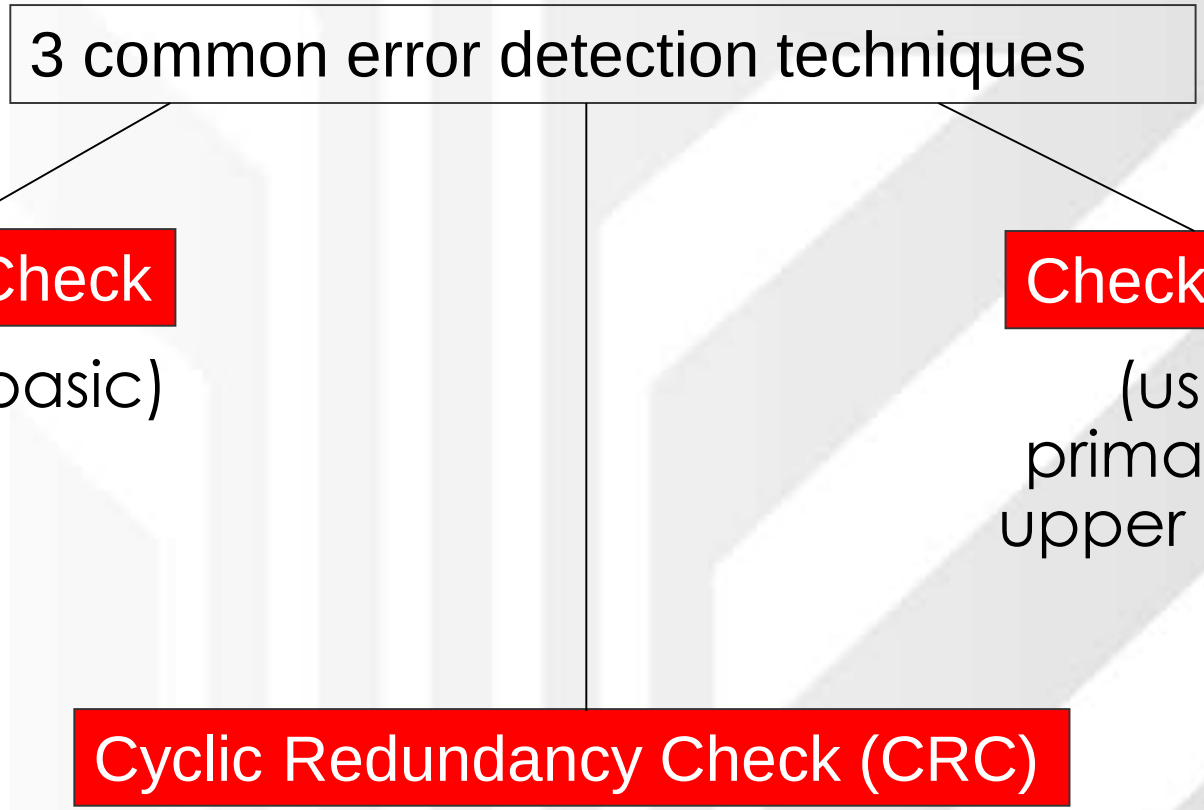
Burst error – two or more bits in the data unit have changed.



Error Detection

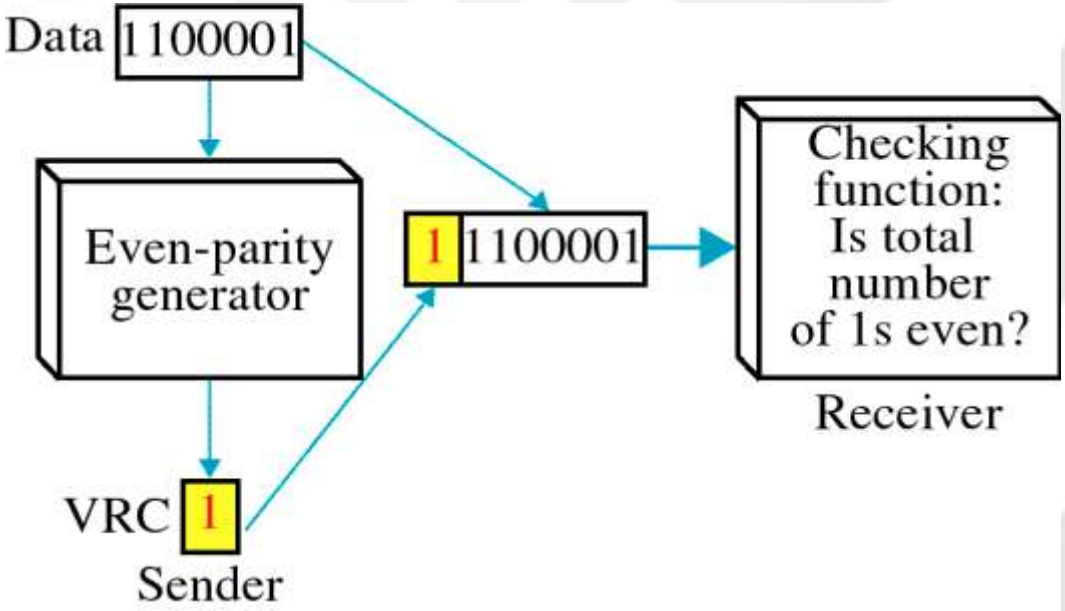
Error detection uses the concepts of redundancy, which means adding extra bits detecting errors at the destination.




3 common error detection techniques
Parity Check
(most basic)
Checksum
(used primarily by upper layers)
Cyclic Redundancy Check (CRC)
(normally implemented in link layer)

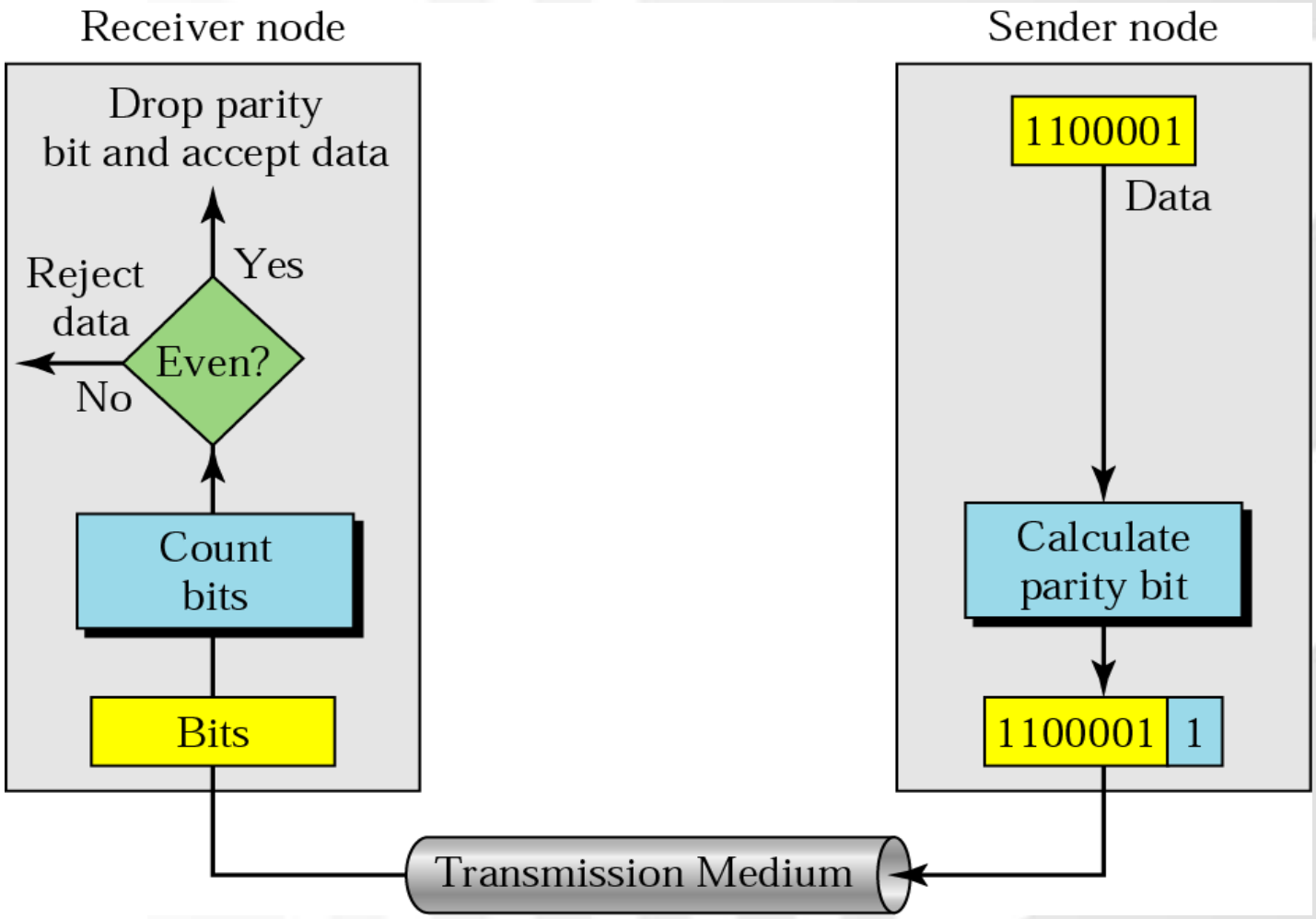
Parity Check

- Simplest technique.
- A redundant bit (parity bit), is appended to every data unit.
- Even parity - the total number of 1's in the data plus parity bit must be an even number.



Even Parity Generator

Data	#1's in data	P	Total # 1's (data and P)
0110110	4 (Even)	0	4 (Even)
0011111	5 (Odd)	1	6 (Even)
0000000	0 (Even)	0	0 (Even)
1010100	3 (Odd)	1	4 (Even)
1111111	7 (Odd)	1	8 (Even)



Example 1

Even Parity

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

11101110 11011110 11100100 11011000 11001001

Example 2

Even Parity

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000 11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data are accepted.

Example 3

Even Parity

Now suppose the word world in Example 1 is corrupted during transmission.

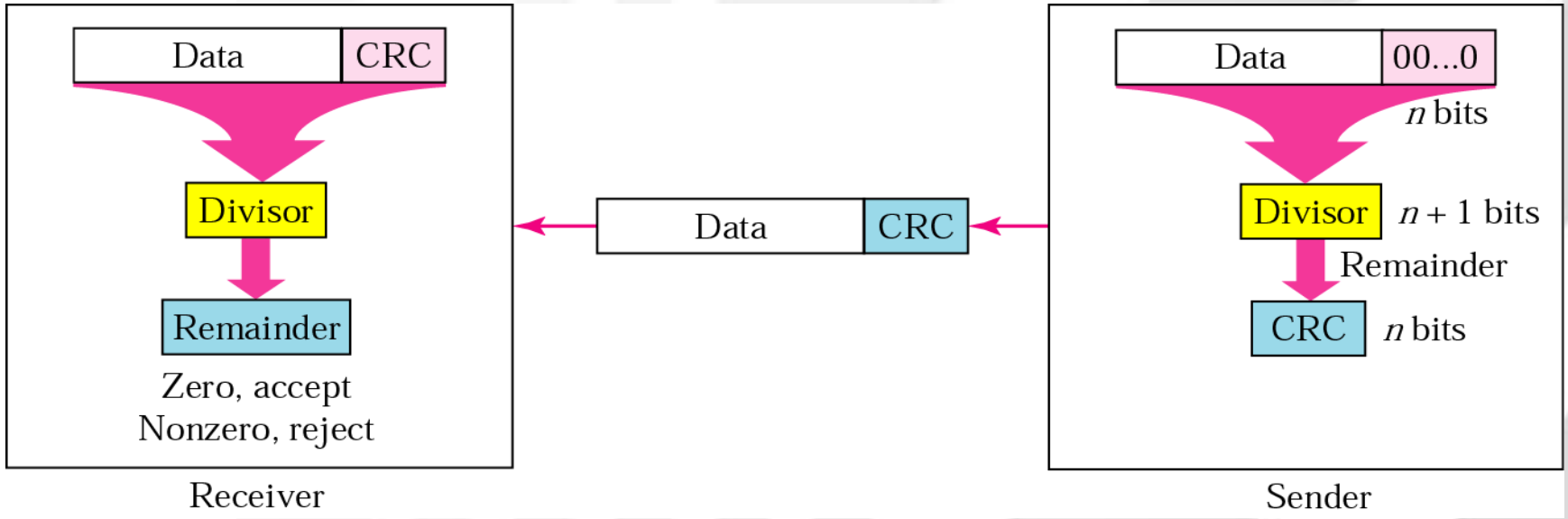
11111110 11011110 11101100 11011000 11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

CRC

- The most powerful of the redundancy checking technique.
- Based on binary division.
- The redundancy bits used by CRC are derived by dividing the data unit by a predetermined divisor; the remainder is the CRC.
- A CRC must:
  - have exactly one less bit than the divisor
  - appending it to the end of the data string must make the resulting bit sequence exactly divisible by the divisor.

CRC generator and checker



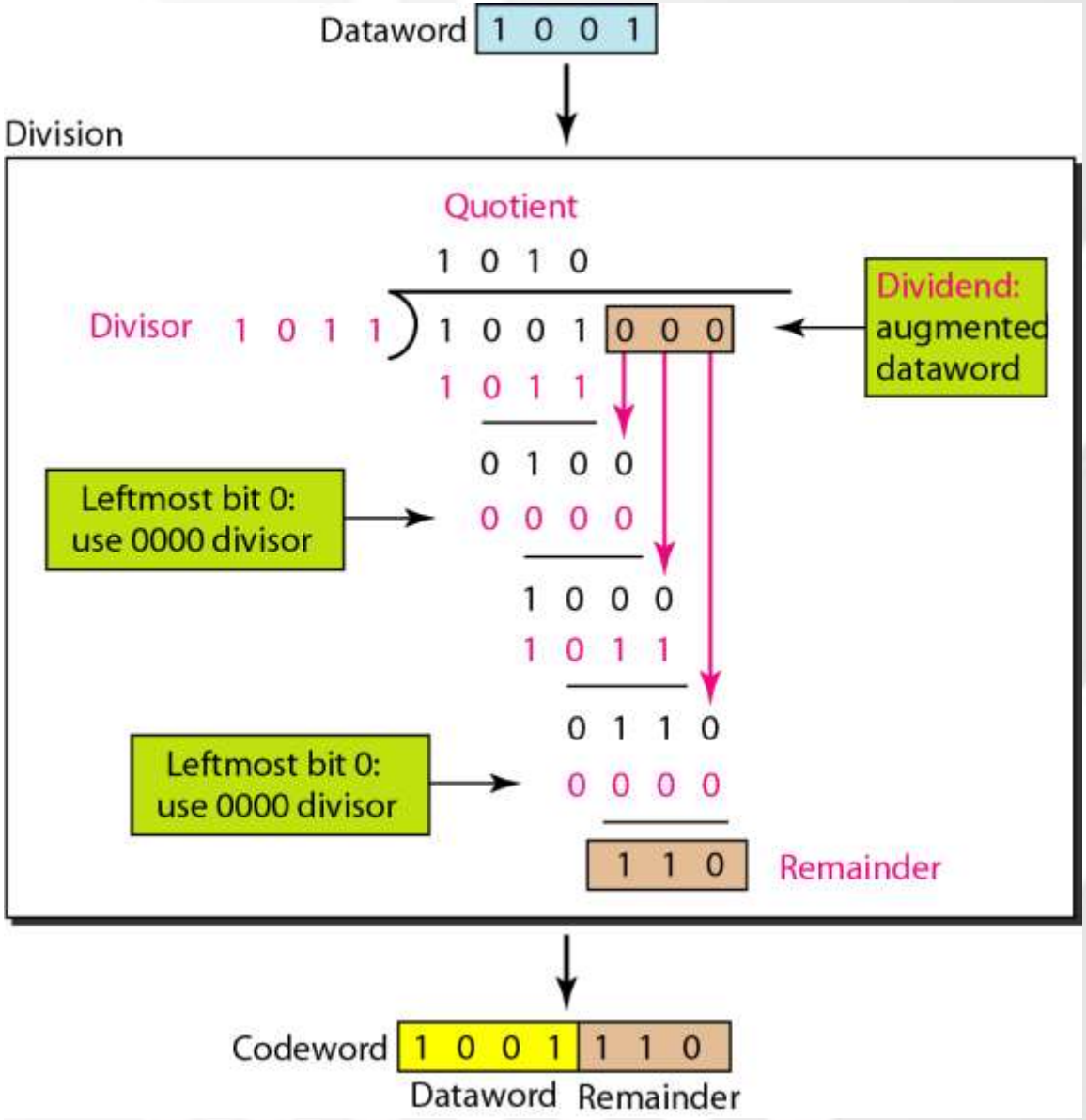
1. Receive the frame.
2. Divide it by divisor.
3. Check the reminder.

1. Get the raw frame.
2. Left shift the raw frame by n bits and divide it by divisor.
3. The remainder is the CRC bit.
4. Append the CRC bit to the frame and transmit.

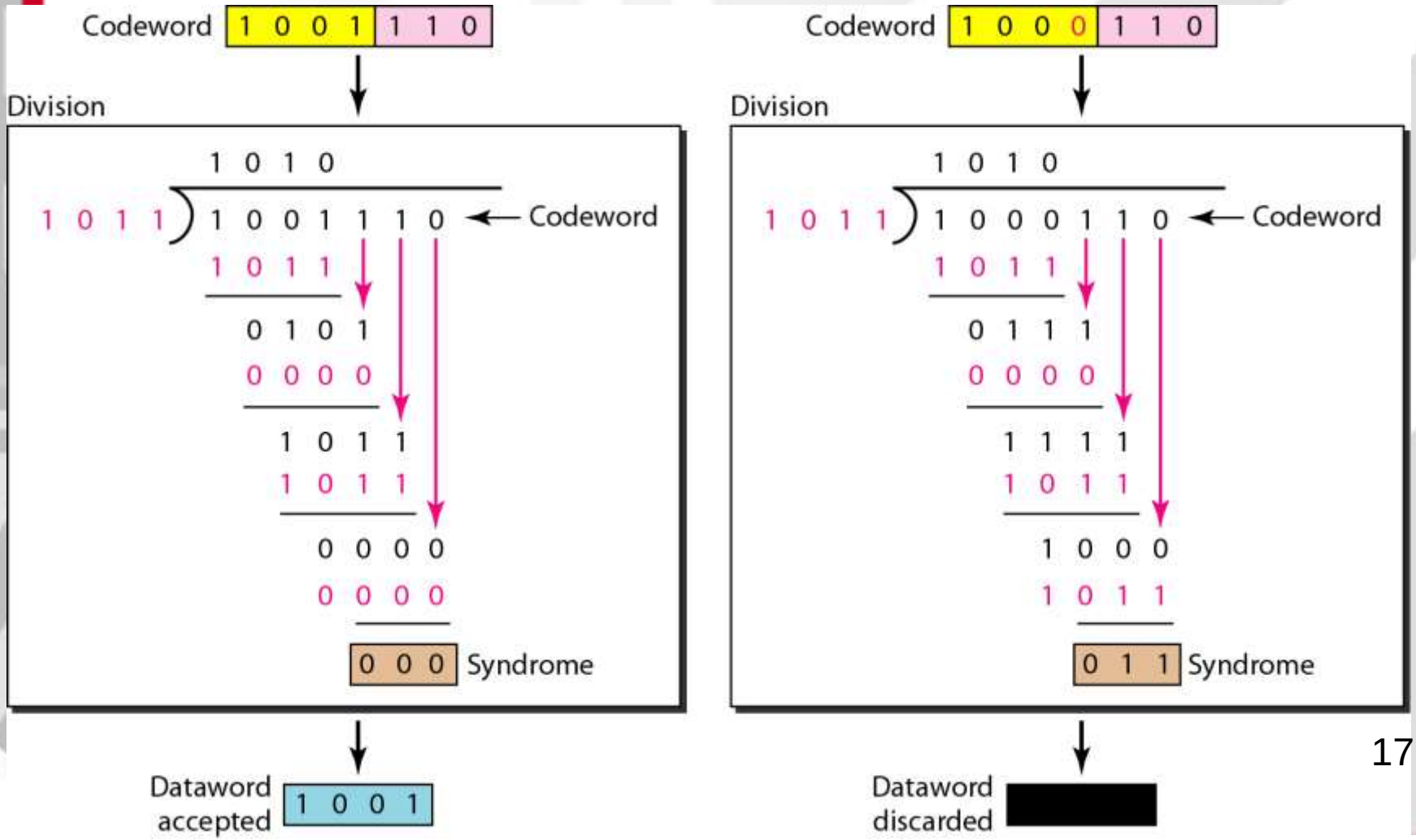
XOR Operation (Exculsive OR)

Input		Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

Division in CRC encoder



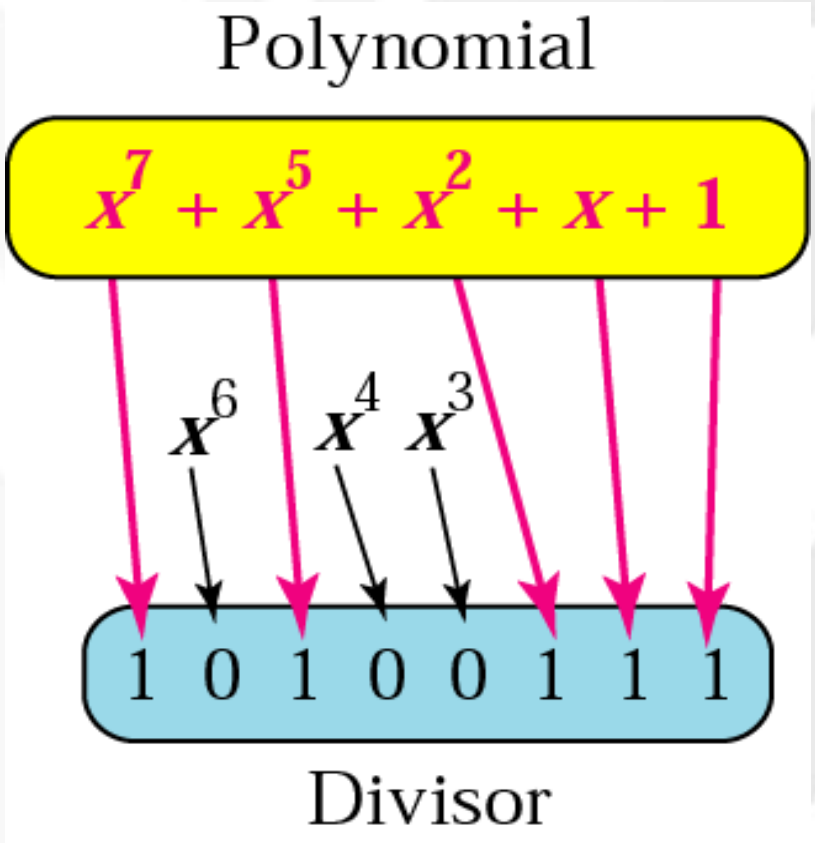
Division in the CRC decoder for two cases



CRC

- CRC generator – at the sending node.
- CRC checker – at the receiving node.
- Polynomial:
  - The CRC generator (the divisor) is most often represented as an algebraic polynomial.
  - e.g.

$$x^7 + x^5 + x^2 + x + 1$$

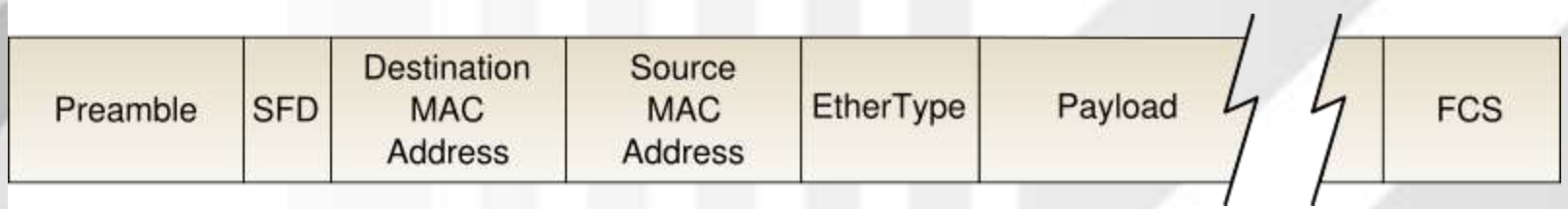


Standard polynomials

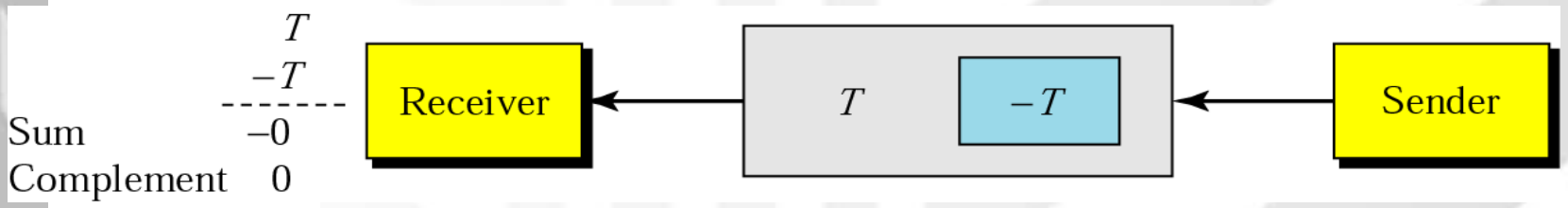
Name	Polynomial	Application
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
ITU-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
ITU-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Checksum

- The error detection used by the higher-layer protocols.
- Check generator – in the sending node
- Checksum checker – at receiving node



Ethernet frame


T
−T
Sum −0
Complement 0
Receiver
T
−T
Sender

Example 6

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

	10101001
	00111001

Sum	11100010
Checksum	00011101

The pattern sent is 10101001 00111001 00011101

Example 7

Now suppose the receiver receives the pattern sent in Example 6 and there is no error.

10101001 00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

	10101001
	00111001
	00011101

Sum	11111111
Complement	00000000 means that the pattern is OK.

Example 8

Now suppose there is a burst error of length 5 that affects 4 bits.

Original data	**10101001 00111001 00011101**
Corrupted data	**10101111 11111001 00011101**

When the receiver adds the three sections, it gets

	10101111
	11111001
	00011101
Partial Sum	**1 11000101**
Carry	**1**
Sum	**11000110**
Complement	**00111001 the pattern is corrupted.**

Error Correction

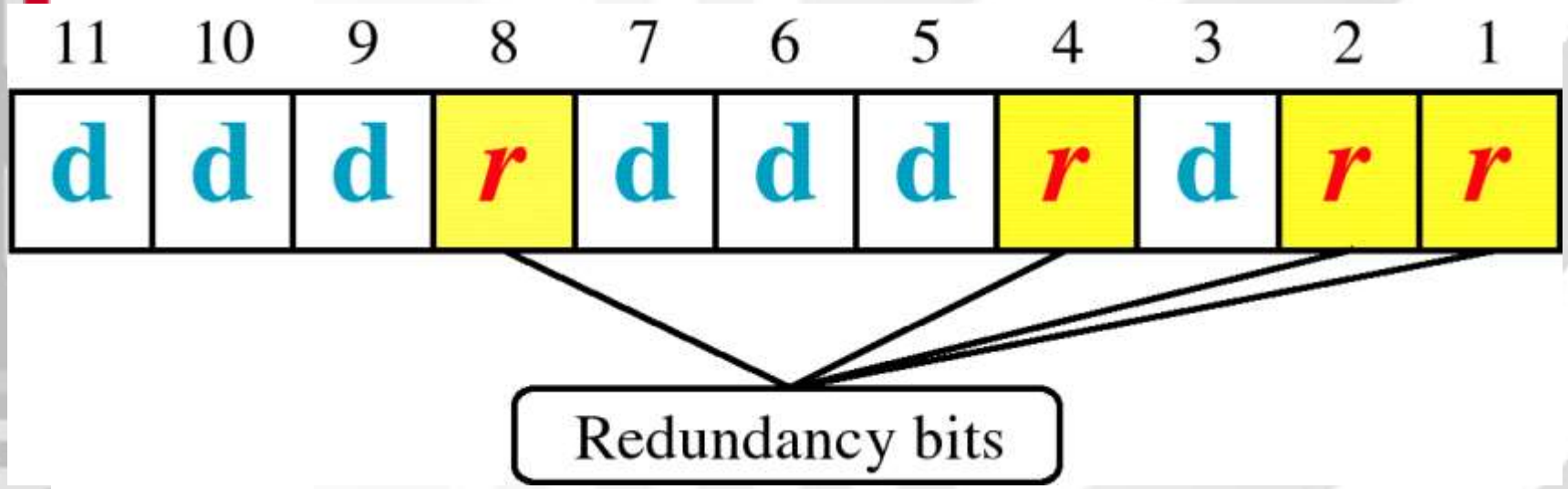
Hamming Code

- Focus on a simple case: Single-Bit Error Correction
- Use the relationship between data and redundancy bits
- Developed by Richard Hamming

Data and redundancy bits

Number of data bits m	Number of redundancy bits r	Total bits m + r
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

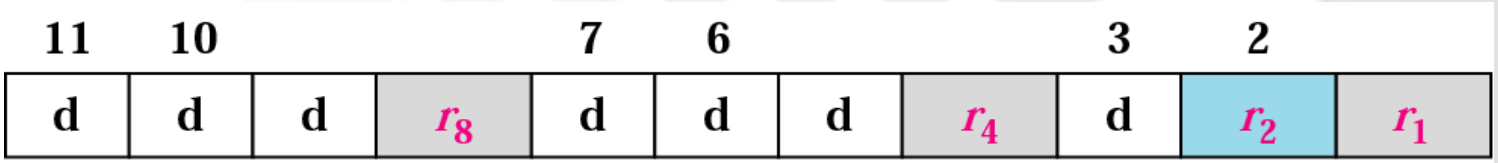
Positions of redundancy bits in Hamming code (11,7)



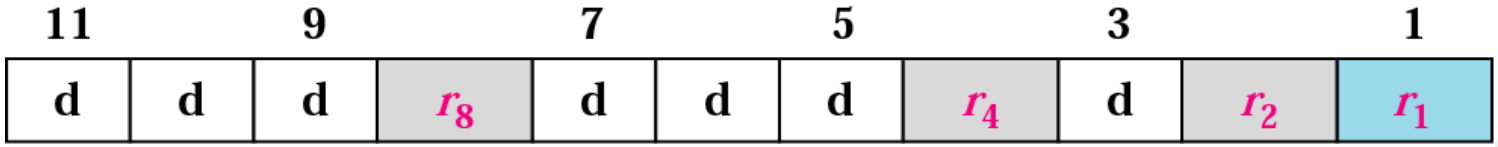
* Check bits occupy positions that are powers of 2

- All bit positions that are powers of 2 are used as parity bits. (positions 1, 2, 4, 8…)
- All other bit positions are for the data to be encoded. (positions 3, 5, 6, 7, 9, 10, 11…)
- Each parity bit calculates the parity for some of the bits in the code word. The position of the parity bit determines the sequence of bits that it alternately checks and skips.
- General rule for position *n*: skip *n*−1 bits, check *n* bits, skip *n* bits, check *n* bits...

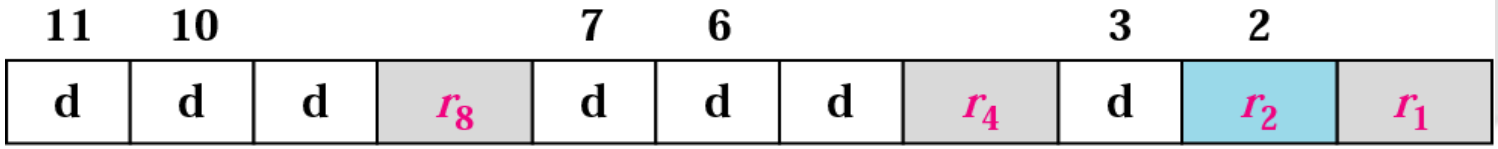
- Position 1 (n=1): skip 0 bit (0=n−1), check 1 bit (n), skip 1 bit (n), check 1 bit (n), skip 1 bit (n), etc. (1,3,5,7,9,11...)
- Position 2 (n=2): skip 1 bit (1=n−1), check 2 bits (n), skip 2 bits (n), check 2 bits (n), skip 2 bits (n), etc. (2,3,6,7,10,11...)
- Position 4 (n=4): skip 3 bits (3=n−1), check 4 bits (n), skip 4 bits (n), check 4 bits (n), skip 4 bits (n), etc. (4,5,6,7,12...)
- Position 8 (n=8): skip 7 bits (7=n−1), check 8 bits (n), skip 8 bits (n), check 8 bits (n), skip 8 bits (n), etc. (8-15,24-31,40-47,...)



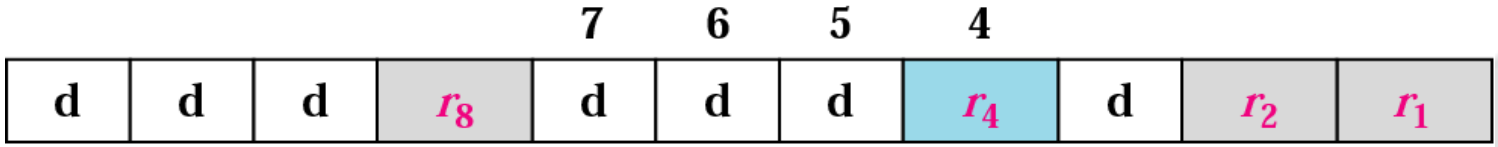
r_1 will take care of these bits.



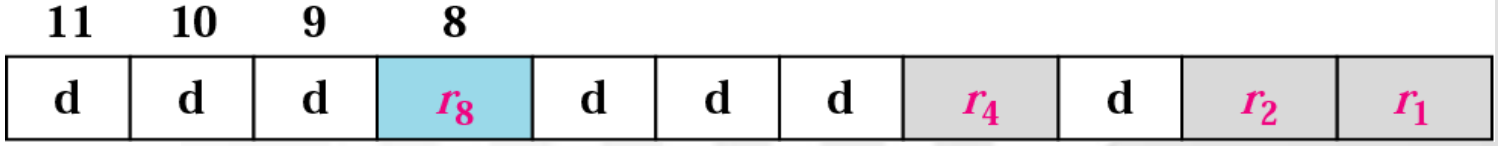
r_2 will take care of these bits.



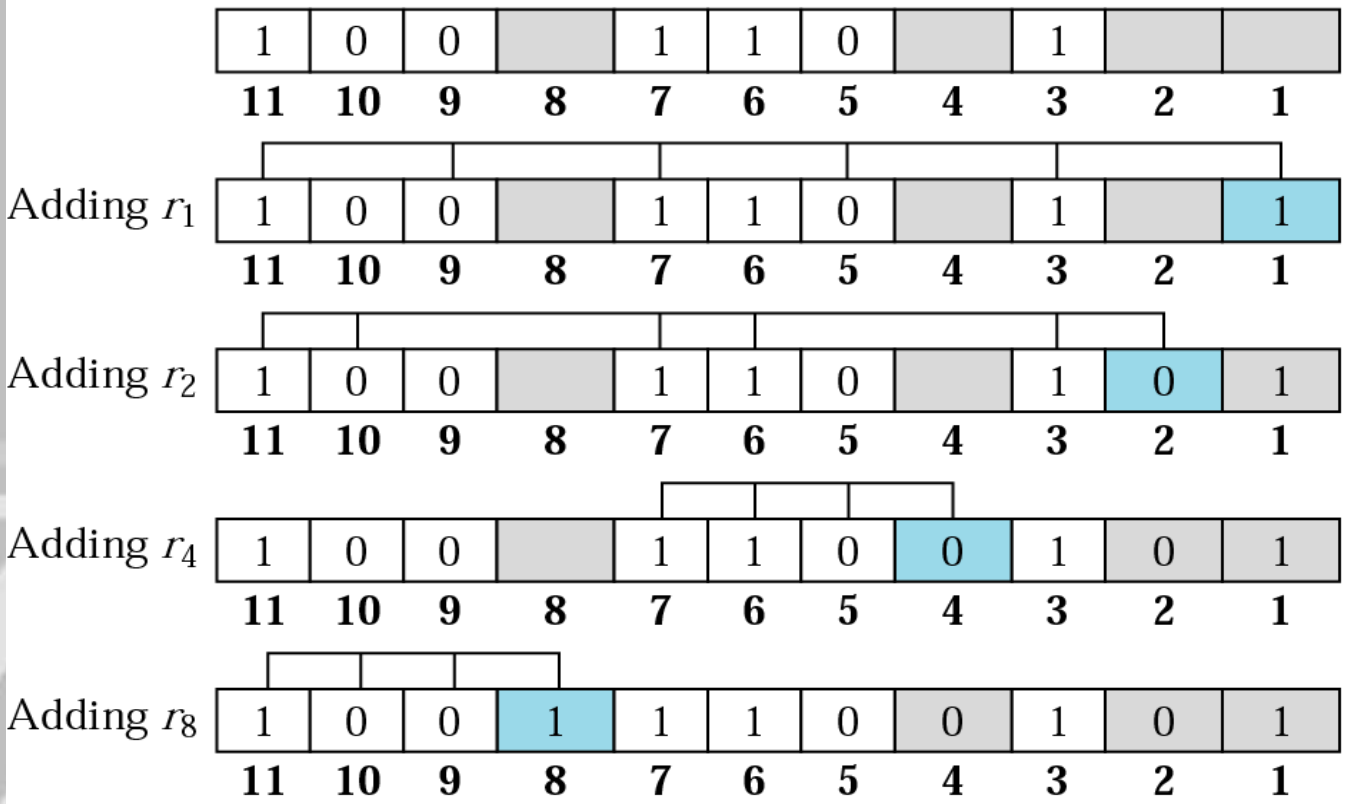
r_4 will take care of these bits.



r_8 will take care of these bits.



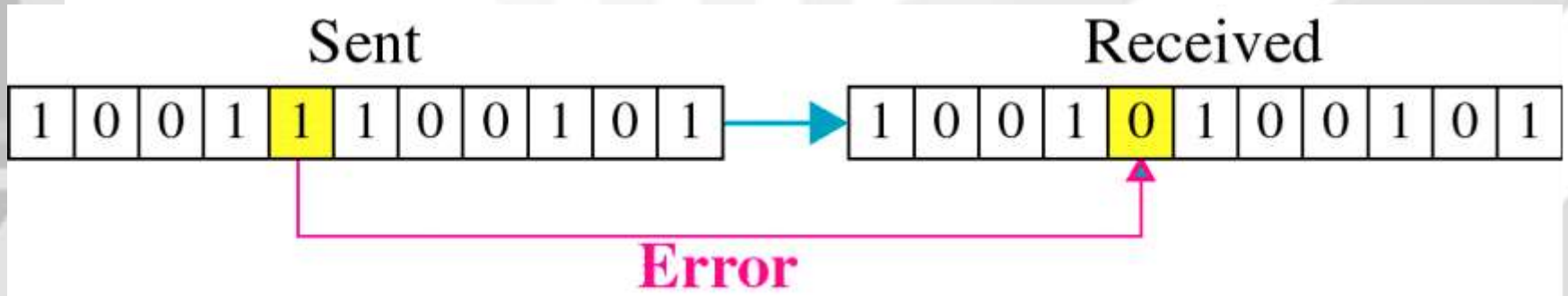
Example of redundancy bit calculation



Data:
1 0 0 1 1 0 1

Code:
1 0 0 1 1 1 0 0 1 0 1

Single-Bit Error



Error Detection